Introduction

This assignment aims to implement an AI agent that analysis several bill images and a single user query, decide the type of user query, and only answer two types of user queries:

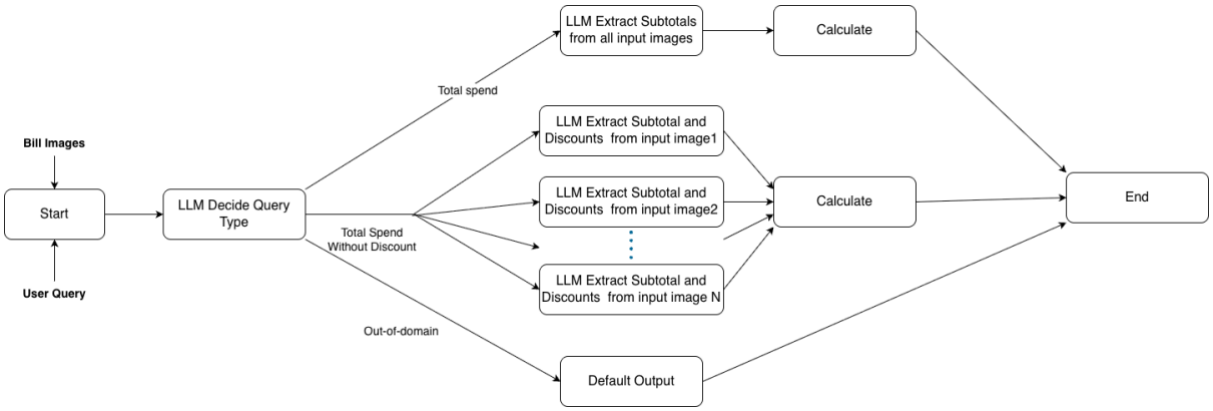
1. How much money did I spend in total for these bills?
2. How much would I have had to pay without the discount?

Any other out-of-domain query will be rejected.

This solution uses **gemini-2.5-flash** as the LLM for performing OCR and natural language processing. For application framework, the solution uses **LangChain** to design a standardized agentic flow.

This report will document the core design, implementation details and test result.

Solution Overview



The solution follows an agentic flow as the diagram, which consists of the following components.

- **Input:** The agent takes the local paths of input images of bills, and the user query in text.
- **Decision Routing:** Before processing the input images, the user query will be passed to LLM to analysis the intention, and route to the correct chain for subsequent processing.
- **Q1 Chain (Total spend):** If the user query is about total spend of bills, the agent will encode and pass all images to the LLM in a single request, which extracts the subtotal fields from the images and calculate the total spend.
- **Q2 Chain (Total spend without discount):** If the user query is about total spend without discount the agent will encode the images and send each of them to the LLM separately, which extracts the subtotals and discounts from the images and calculate the total spend without discount.
- **Out-of-domain Query:** If the user query is identified as irrelevant, the agent will simplify respond with pre-defined text.

Implementation Details

Image Encoding

In order to leverage the multimodal capability of LLM, images are directly sent to the LLM after encoding, instead of preprocess with additional OCR services. In this solution, images are encoded in base64 format and convert into data URLs using the *base64* and *mimetypes* libraries in Python. Then, the URLs are passed as part of chat messages by indicating the type as *image_url*, along with other text prompts.

Routing

In order to understanding the intention of the user query and route the requesting using the correct chain (q1, q2, rejected), the user query is first passed to the LLM using the prompt below.

```
You are a precise query classifier for bill/receipt totals
only.
Treat 'receipt', 'receipts', 'invoice', 'invoices' same as
'bill', 'bills'.
Context: There are {total_count} bills/receipts in input.

**q1**: COMPLETE sum of ALL spending across ALL input
bill(s)/receipt(s) (actual total paid)
**q2**: COMPLETE sum of ALL spending across ALL input
bill(s)/receipt(s) WITHOUT discounts

**ALLOW q1/q2** if:
- No bill number specified OR matches {total_count}  ("these
7 bills" when {total_count} =7)
- "total spend", "sum of all/spending", "total amount" +
bills/receipts

**STRICT BLOCK** (other) if:
- Specific bill ("first", "second", "this one", "bill 2")
- Wrong count ("these 3 bills" when {total_count} =7)
- Products (milk, groceries)
- Line items

**ONLY** output: `q1`, `q2`, or `other`
```

If the user request match q1/q2, it will then invoke the corresponding chain. If unmatched, it will return a predefined string response without further generation.

Q1 Chain (Total spend)

In order to calculate the total, spend from the bills, the chain first will ask the LLM to extract the subtotal fields from every images in one single request, as this task is relatively easy. After obtaining the structured output, the chain calculates the sum by python calculation.

Prompt:

```
You need to find the ROUNDED SUBTOTAL from the receipt
images.
              You must follow the rules below:
              1. You NEED to extract from EVERY images and
include in the response
              2. The subtotal should be the number ACTUALLY
paid, usually after rounding and discount.
              3. ONLY record information from the images, DO
NOT make assumptions or facts.
              4. ONLY output the subtotal, do not output any
other things other than the numeric value of subtotal
              5. if no value found, output 0
              6. You are given {len( x['images'])} images,
you MUST handle them separately
```

Output Structure:

```
class Q1_Response(BaseModel):
    subtotals:List[float]
```

Q2 Chain (Total spend without discount)

In order to calculate the total, spend from the bills without discount, the chain first will ask the LLM to extract the subtotal and discount fields of the images as this task. As this task is more complex, each image will be process by separate LLM request separately to increase accuracy. After obtaining the structured output, the chain calculates the sum by python calculation.

Prompt:

```
Do the following:
          1. Extract the subtotal, it should be the number
BEFORE rounding but AFTER discount.
          2. get the discounts in monetary value of the image
          3. For discount, DO NOT include rounding, total
discount, or remaining balance
```

Output Structure:

```
class Q2_Response(BaseModel):
  subtotal:float
  discount:List[float]
```

Testing Result

The agent is tested with several custom test cases.

Query Type	Number of test cases	Accuracy
Q1 (total spend)	3	100%
Q2 (total spend without discount)	2	100%
Out of domain	5	100%

Conclusion

In conclusion, this project delivers an AI agent that reliably analyzes multiple supermarket bill images and answers two well-defined financial queries: the total amount spent and the hypothetical total without discounts. On the custom test cases constructed for this homework, the system achieves 100% accuracy, both in classifying query types and in computing the corresponding numeric outputs. Nonetheless, its capabilities are constrained by several limitations: it depends on clear and legible receipt images, and its performance may degrade when handling a large number of high-resolution images due to the underlying model's finite context window and input size constraints.

GitHub Repository

The full implementation and this report are available at: https://github.com/TommyS725/FTEC5660